GOLD: Threads (Wed Oct 14)

CPU Cores

Most of your cores are asleep

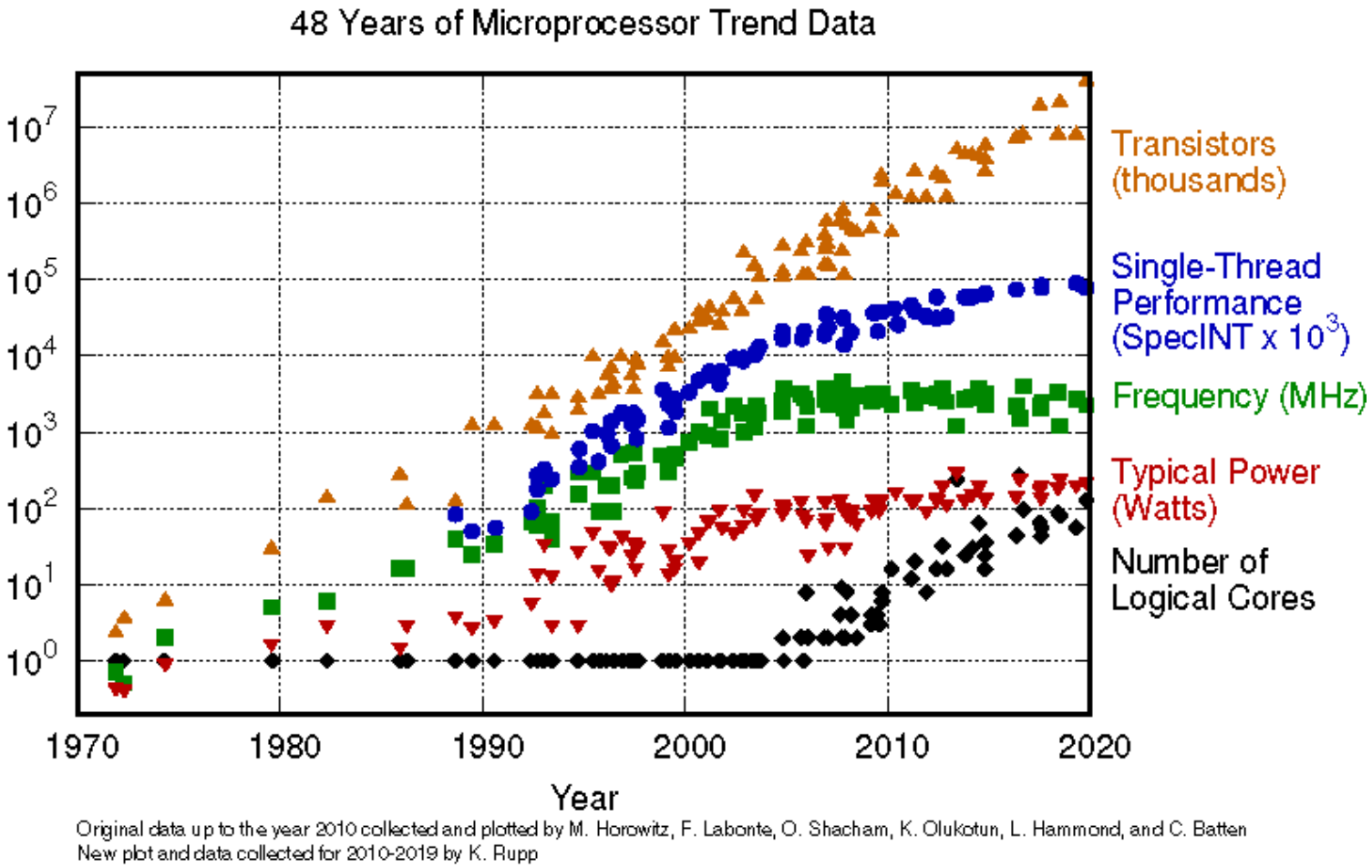
The machine running this slide has several cores; your laptop probably has eight or more.

This means 8 places where work can happen at the same instant.

Every program we have written so far in this course used exactly one of them.

The others sat idle, at least as far as running Java programs goes.

Why not one really big core?



Around 2005, chips hit a power and heat wall:

- **heat** grows faster than the number of transistors, and no way to safely dissipate that much heat
- **speed limits** on electrical signals: even at half the speed of light, data can only move 1 mm per clock cycle at 4 GHz
- they couldn't run faster, so they grew wider
- Since 2005, processors have included more cores at the same speed.

The takeaways:

- Free speed from hardware ended ~20 years ago.
- Using the width (cores) is the **software's** job. That's this week.

How many cores do you have?

- Run `nproc` in the terminal
- Go to Windows Task Manager → Performance → CPU
- Go to Mac Task Manager → About This Mac

Introducing Multithreading

Multithreading Demo

Multithreading is how we can get Java to execute two pieces of code at the exact same time (using multiple cores).

Let's explore these ideas together first:

```
class Main {
    void main() {
        for (int j = 1; j <= 4; j++) {
            Thread t = new Thread(new MyTask(j));
            t.start();                                  // run() begins on a new thread
        }
        IO.println("main: all four started");
    }
}

class MyTask implements Runnable {
    private int number;

    MyTask(int number) { this.number = number; }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            IO.println(i + " from thread #" + number);
            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                return;
            }
        }
    }
}
```

Our implementation of the `Runnable` interface parallels the multi-character Nim Game closely:

Compare:

Previously: the Multiplayer Game	Today: threads

`interface Player { int takeSome(...); }`	`interface Runnable { void run(); }`
`class Dragon implements Player { ... }`	`class MyTask implements Runnable { ... }`
`new Game(dragon, butterfly)`	`new Thread(task)`
`game.play()` calls **your** `takeSome`	`t.start()` calls **your** `run` **on a new thread**

The bottom-right cell is entirely new however: we never define `start()`.

The JVM doesn't just call your `run()` method; it calls it in parallel with you.

Demo: Loops and Prints on Threads

The multithreading demo from the slides, as a workspace you can experiment with.

Things to try, one at a time, and **Run** after each:

- Change the sleep to `0`, then to `1000`.
- Start ten threads instead of four.
- Make `main` print something *after* the loop, then move that line *before* the loop.
- Throw a `RuntimeException` inside `run()` for thread #2 only, and watch the other threads.

Threads

The Thread Ceremony

Step 1: Create a class that implements the Runnable interface.

Your code goes in `run()`

```
public class MyTask implements Runnable {   // 1. sign the contract
    @Override
    public void run() {                      //    ...the one promised method
        System.out.println("working on: " + Thread.currentThread().getName());
    }
}
```

Step 2: Construct a new Thread object with your task object

```
Thread t = new Thread(new MyTask());
```

Step 3: Start the thread

```
t.start();
```

The worker begins executing `run()` alongside your code which continues

All together now:

```
class Main {
    void main() {
        Thread t = new Thread(new MyTask());      // 2. hire a worker, hand it the tas
        t.start();                                // 3. GO: run() begins on the new th
        IO.println("main is on: " + Thread.currentThread().getName());
    }
}

class MyTask implements Runnable {                // 1. sign the contract
    @Override
    public void run() {                           //    the one promised method
        IO.println("working on: " + Thread.currentThread().getName());
    }
}
```

What Is a Thread?

What is a Thread though?

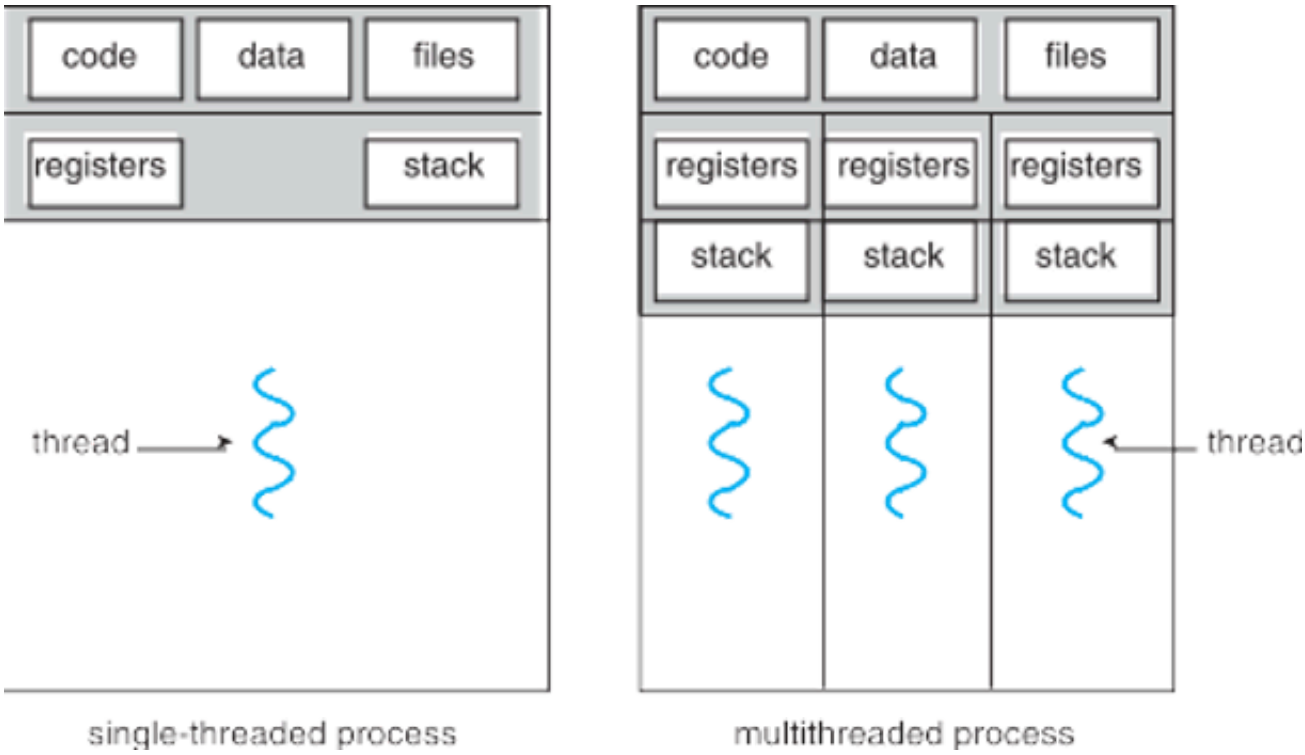
A Thread is an independent path of code execution, and is managed by the JVM.

Each thread has separate:

- location in the code
- its own variables and stack

Threads share the same:

- heap (where all the objects live)
- sharing objects between threads is the root of a lot of danger...



Memory Models

There are two models:

Shared memory architecture (Java)

- Single program

- Multiple threads share memory space (variables)

Message passing

- Messages sent between separate processes
- One process per program (generally)
- May run on different physical computers

Threads are somewhat independent

An Exception in one thread does not bring everything down:

```
class Main {
    void main() {
        for (int j = 1; j <= 3; j++) {
            new Thread(new MyTask(j)).start();
        }
        IO.println("main: done starting");
    }
}

class MyTask implements Runnable {
    private int number;

    MyTask(int number) { this.number = number; }

    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            if (number == 2 && i == 1) {
                throw new RuntimeException("thread #2 gives up");
            }
            IO.println(i + " from thread #" + number);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                return;
            }
        }
    }
}
```

Non-determinism

Two workers again

```
class Main {
    void main() throws InterruptedException {
        Thread t1 = new Thread(new Interleave('a'));
        Thread t2 = new Thread(new Interleave('b'));
        t1.start();
        t2.start();
        t1.join();                                  // wait for both (explained in a fe
        t2.join();
        IO.println();
    }
}

class Interleave implements Runnable {
    private char c;
    private double sink = 0;

    Interleave(char c) { this.c = c; }

    @Override
    public void run() {
        for (int i = 0; i < 60; i++) {
            IO.print(c);
            for (int j = 0; j < 4000; j++) {
                sink += Math.hypot(i, j);           // busy work between prints
            }
        }
        if (sink == -1) IO.print("!");              // keeps sink alive
    }
}
```

Pause and predict (poll time)

Q. Two workers: one prints `a` sixty times, the other `b`, with busy work between print calls. What does the output look like?

- **A.** `aaaa…` then `bbbb…`: the first thread finishes before the second starts
- **B.** `abababab…`: the two threads take turns
- **C.** A scrambled mix, the same mix every run
- **D.** A scrambled mix, and a different mix every run

Run It Twice

Run it twice

The same program. Press Run, then press Run again, and compare the two lines.

```
class Main {
    void main() throws InterruptedException {
        Thread t1 = new Thread(new Interleave('a'));
        Thread t2 = new Thread(new Interleave('b'));
        t1.start();
        t2.start();
        t1.join();                                  // wait for both (explained in a fe
        t2.join();
        IO.println();
    }
}

class Interleave implements Runnable {
    private char c;
    private double sink = 0;

    Interleave(char c) { this.c = c; }

    @Override
    public void run() {
        for (int i = 0; i < 60; i++) {
            IO.print(c);
            for (int j = 0; j < 4000; j++) {
                sink += Math.hypot(i, j);           // busy work between prints
            }
        }
        if (sink == -1) IO.print("!");              // keeps sink alive
    }
}
```

The OS thread scheduler decides which thread runs when, not your code

- The scheduler decides differently every time.
- That is not a bug to fix; this is inevitable.
- When things can happen at the same time, we lose control over the exact order.

run() Is Not start()

Watch this fail: `run()` is not `start()`

It's tempting: you wrote `run()`, so... why not call it?

```
class Main {
    void main() throws InterruptedException {
        IO.println("main is running on:    " + Thread.currentThread().getName());

        RunVsStart task = new RunVsStart();
        task.run();

        Thread t = new Thread(task);
        t.start();
        t.join();
    }
}

class RunVsStart implements Runnable {
    @Override
    public void run() {
        IO.println("run() is executing on: " + Thread.currentThread().getName());
    }
}
```

Pause and predict (poll time)

Q. The first line printed is `main is running on: main`. What is the **second** line?

- **A.** `run() is executing on: main`: a plain method call, no new thread
- **B.** `run() is executing on: Thread-0`: calling `run()` starts the thread
- **C.** Compile error: `run()` may only be called by `start()`
- **D.** The two lines race and print in either order

sleep

`sleep`: pausing one worker

```
Thread.sleep(int time)
```

The Thread class has a static method `sleep(time)` which puts the current thread to rest for (approx. `time` ms; nothing is exact).

Any method that causes the current thread to go into a non-active state must catch (or use `throws`) the `InterruptedException`

```
class Main {
    void main() {
        new Thread(new SleepCountdown()).start();
        IO.println("main: I'm not waiting around.");
        IO.println("main: done.");
    }
}

class SleepCountdown implements Runnable {
    @Override
    public void run() {
        for (int i = 3; i >= 1; i--) {
            IO.println("worker: " + i + "...");
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                return;                          // asked to stop: comply
            }
        }
        IO.println("worker: liftoff!");
    }
}
```

Output (the worker's first line can land before or after main's two; the countdown always ends last):

```
main: I'm not waiting around.
main: done.
worker: 3...
worker: 2...
worker: 1...
worker: liftoff!
```

Why does Java have `InterruptedException`?

- To allow threads to wake each other up as needed.
- You can't get input if you're asleep ....
- ... so this is another example of exceptions as out-of-band signalling

join

Watch this fail: reading the answer too early

A worker counts to 200 million. `main` starts it and then prints the worker's total.

```
class Main {
    void main() {
        Summer task = new Summer();
        Thread t = new Thread(task);
        t.start();
        IO.println("The thread has started counting.");
        IO.println("total = " + task.total);       // the worker may still be running
    }
}

class Summer implements Runnable {
    long total = 0;

    @Override
    public void run() {
        for (long i = 1; i <= 200_000_000L; i++) {
            total += 1;
        }
    }
}
```

Pause and predict (poll time)

Q. What does it print?

- **A.** `total = 200000000` every time: the worker finishes before `main` reads
- **B.** Sometimes `total = 0`, sometimes a partial count
- **C.** `total = 0` every time
- **D.** Compile error: `total` belongs to the worker

join: Patience Please

Patience please: `join`

The `join()` method is a simple kind of synchronization: let's wait for *another thread* to finish before continuing the current one.

For a Thread t, calling

```
t.join();
```

Blocks the “current thread”, which is the one that called `t.join()`, until Thread t completes (returns from `run()`)

- It's easy to get this backwards
- Think `t.join()` as meaning "wait for `t` to finish"

`join()` may throw an `InterruptedException`, so it is generally is used in try-catch clause

```
class Main {
    void main() {
        Summer task = new Summer();
        Thread t = new Thread(task);
        t.start();
        try {
            t.join();                                // wait for t to finish, then read
        } catch (InterruptedException e) {
            return;
        }
        IO.println("total = " + task.total);
    }
}

class Summer implements Runnable {
    long total = 0;

    @Override
    public void run() {
        for (long i = 1; i <= 200_000_000L; i++) {
            total += 1;
        }
    }
}
```

Decomposition

Two ways to carve up the work

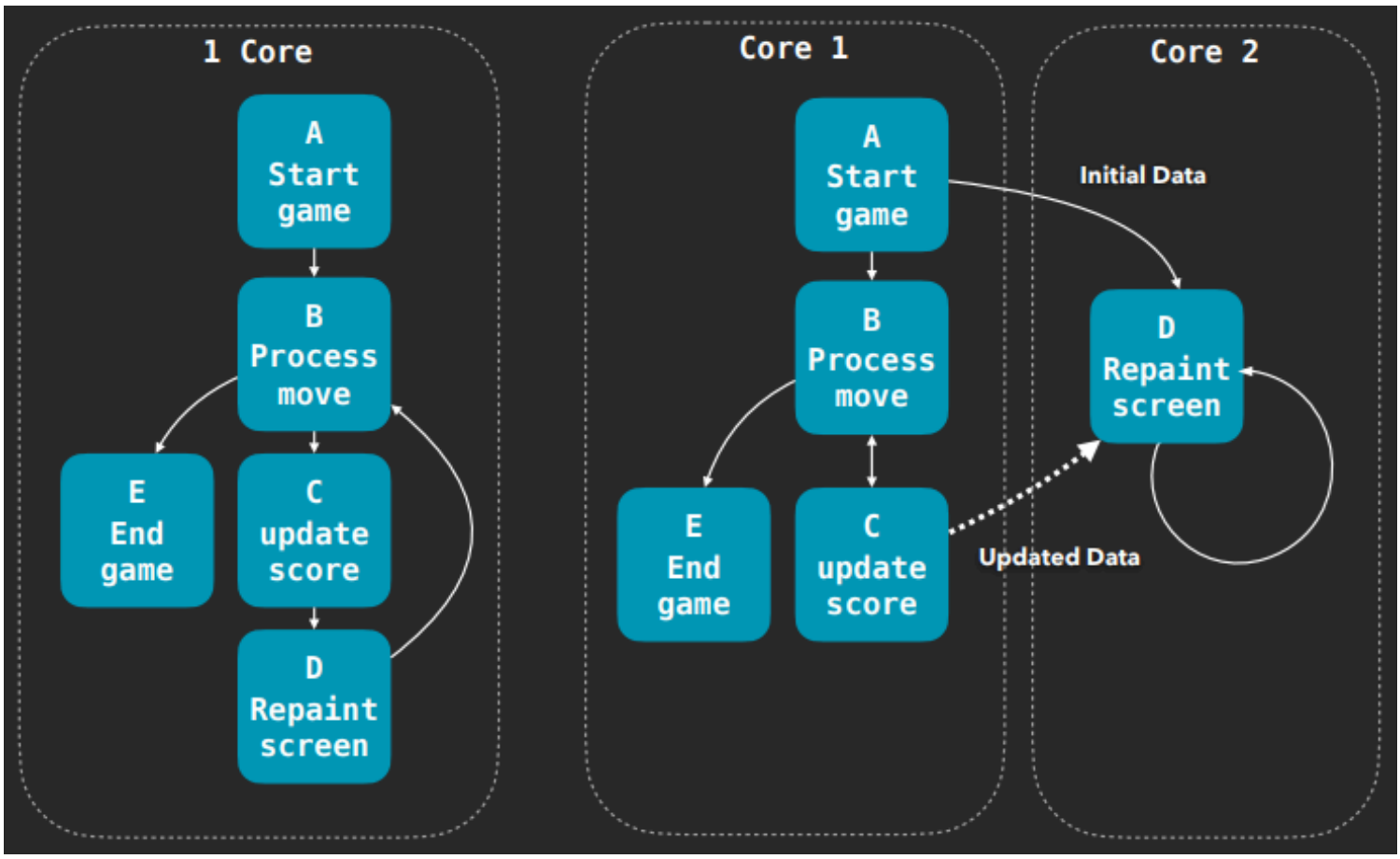
In order to use multiple cores effectively, we have to give them both some of the work.

How do you break down a large problem into pieces?

We call this *decomposing* the problem

Two approaches to decomposition:

Task decomposition: different *jobs* (different code) to different workers:



The game logic and the screen repainting run as separate tasks.

Benefit: the game stays **responsive** even when the logic is busy.

You can apply this in your own projects:

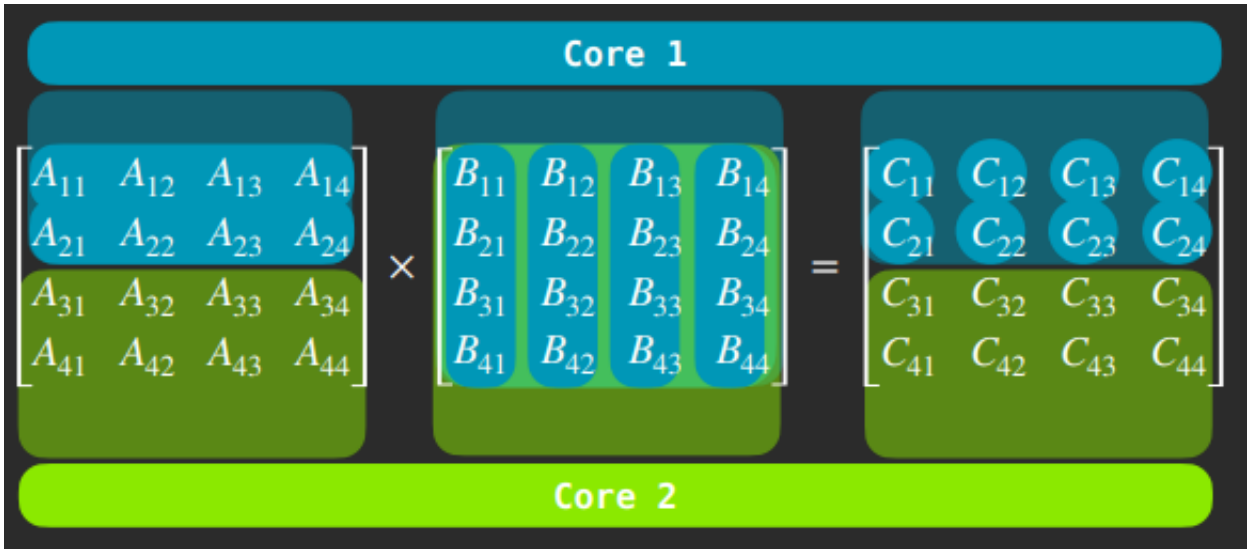
- Think of splitting up by the *kind of work* to be done
- Often, different kinds of work can be done independently

- The primary benefit is responsiveness

Domain decomposition: same job, different *slices of the data*:

Matrix Multiplication

- None of the entries of the matrix product depend on other parts of the answer.
- Two cores can compute half of the product at the same time.
- We partition the first input A into two halves.



Each subtask runs the

- *same code* but
- *on different input*

Each subdomain is given to a task running on a different core, if available

Primary benefit: raw speed

Primary drawback: not always possible, may require a deep understanding of our problem

Every worker runs the **same code** on its own slice.

Payoff: **raw speed,** which is a claim we should test.

Measuring the Speedup

Measuring the Speedup: Counting Primes

I can't claim that some approach gives you raw speed without some evidence.

Let's try to measure how long it takes to count all of the prime numbers up to some point.

Our thread task will be to check if numbers are prime:

```
public class Speedup implements Runnable {
    private int lo, hi;
    private int count = 0;                    // each worker's PRIVATE tally

    public Speedup(int lo, int hi) { this.lo = lo; this.hi = hi; }

    public void run() {
        for (int n = lo; n < hi; n++)
            if (isPrime(n)) count++;
    }

    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int d = 2; d * d <= n; d++)
            if (n % d == 0) return false;
        return true;
    }
    ......
```

Let's setup a test.

First, a solo worker on the whole range:

```
int LIMIT = 8_000_000;

// --- one worker does everything ---
long start = System.currentTimeMillis();
Speedup solo = new Speedup(0, LIMIT);
solo.run();                               // just a method call, one thread
long soloMs = System.currentTimeMillis() - start;
```

Now let's do it again with four threads:

```
        Speedup[] workers = new Speedup[4];
        Thread[] threads = new Thread[4];
        int chunk = LIMIT / 4;
        for (int i = 0; i < 4; i++) {
            workers[i] = new Speedup(i * chunk, (i + 1) * chunk);
            threads[i] = new Thread(workers[i]);
            threads[i].start();
        }
        for (int i = 0; i < 4; i++)
            threads[i].join();
```

Let's put it all together:

```
class Main {
    void main() throws InterruptedException {
        int LIMIT = 80_000_000;

        // --- one worker does everything ---
        long start = System.currentTimeMillis();
        Speedup solo = new Speedup(0, LIMIT);
        solo.run();                                  // just a method call: one thread
        long soloMs = System.currentTimeMillis() - start;
        IO.println("1 thread:  " + solo.count + " primes in " + soloMs + " ms");

        // --- four workers, a quarter of the range each ---
        start = System.currentTimeMillis();
        Speedup[] workers = new Speedup[4];
        Thread[] threads = new Thread[4];
        int chunk = LIMIT / 4;
        for (int i = 0; i < 4; i++) {
            workers[i] = new Speedup(i * chunk, (i + 1) * chunk);
            threads[i] = new Thread(workers[i]);
            threads[i].start();
        }
        int total = 0;
        for (int i = 0; i < 4; i++) {
            threads[i].join();
            total += workers[i].count;
        }
        long fourMs = System.currentTimeMillis() - start;
        IO.println("4 threads: " + total + " primes in " + fourMs + " ms");
        IO.println(String.format("speedup: %.1fx", (double) soloMs / fourMs));
    }
}
```

```
class Speedup implements Runnable {
    private int lo, hi;
    int count = 0;                                  // each worker's PRIVATE tally

    Speedup(int lo, int hi) { this.lo = lo; this.hi = hi; }

    @Override
    public void run() {
        for (int n = lo; n < hi; n++) {
            if (isPrime(n)) count++;
        }
    }

    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int d = 2; d * d <= n; d++) {
            if (n % d == 0) return false;
        }
        return true;
    }
}
```

Example output:

```
1 thread:  539777 primes in 4616 ms
4 threads: 539777 primes in 2042 ms
speedup: 2.3x
```

Why is this not 4x faster?

On Ed, we get roughly 2x faster. Why?

- The workload isn't evenly distributed.
  - The largest 1/4 of numbers are harder than the smallest 1/4.
  - We could distribute it more fairly, but it's not trivial.
- Hardware factors: four busy cores have a slower clockspeed than one busy core.
- Machine limitations: The Ed machine may have few cores available or a high load.

Cliffhanger: Two Workers, One Counter

Two workers with one shared `static` counter.

Each adds 1 a million times.

Expected counter value: 2 million

```
class Main {
    void main() throws InterruptedException {
        Thread t1 = new Thread(new RaceCondition());
        Thread t2 = new Thread(new RaceCondition());
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        IO.println("expected 2000000, got " + RaceCondition.counter);
    }
}

class RaceCondition implements Runnable {
    static int counter = 0;                          // one shared box

    @Override
    public void run() {
        for (int i = 0; i < 1_000_000; i++) {
            counter++;
        }
    }
}
```

Pause and predict (poll time)

Q. Each worker adds 1 to the shared counter a million times. What does it print?

- **A.** `2000000` every time
- **B.** `2000000` most of the time, occasionally less
- **C.** Less than `2000000` almost every time, and a different number each run
- **D.** Compile error: two threads may not share a `static` field

Take Home: Parallel Sum

Two files to finish.

- `SliceSummer` is a worker: it adds up one slice of an array, `data[lo]` through `data[hi - 1]`, into its own private `total`.

- `Main.parallelSum` hires four of them, one quarter of the array each, starts them all, waits for all four, and adds their answers.

Part 1: the worker. Finish `run()` in `SliceSummer.java`: a loop from `lo` up to, but not including, `hi`, adding each entry to `total`. Nothing is shared: every worker has its own slice and its own total.

Part 2: the hiring. Finish `parallelSum` in `Main.java`: four `SliceSummer` objects, four `Thread` objects, `start()` all four, then `join()` each one (`join` can throw `InterruptedException`; catch it), then add the four `getTotal()` values.

The order matters: all four started *before* any is joined, or the workers run one after another and nothing is parallel. And every `join` *before* reading a total, or you read a number that is still being computed, as on the join slide.

Run sums eight million numbers and prints `Total: 36000000`.

Press **Test** to run five tests.